Universidad de Antioquia
Facultad de Ingeniería

Informática II – Semestre 2026-2

Desafío II

Informe de análisis y diseño
Ude@Book

Presentado por:

Juan José Arboleda Cardona
Brayan Camilo Silva Porras

Medellín, 9 de octubre de 2026

1 Contexto del problema

Ude@Book es una red social. Nosotros solo tenemos que hacer la parte lógica, o sea, la parte que maneja los datos de los usuarios, sus amigos, mensajes, solicitudes de amistad, álbumes y fotos. Todo se maneja por consola y los datos se cargan desde archivos.

De cada usuario se guarda: nombre de usuario (no se repite), contraseña, correo, ciudad, país, fecha de nacimiento, fecha en que se inscribio y foto de perfil. Ademas, cada usuario tiene:

- Amigos (la amistad es de los dos lados).
- Mensajes enviados y mensajes recibidos, ordenados por fecha. Cada mensaje puede estar leído o no leído.
- Solicitudes de amistad que puede aceptar o rechazar.
- Sugerencias de amistad, que son los amigos de sus amigos ordenados por cuantos amigos tienen en común.
- Álbumes que pueden ser públicos, privados o solo para amigos. Cada álbum tiene fotos y cada foto tiene fecha, tamaño, ruta y likes.

Para "ver" una foto solo hay que mostrar la ruta del archivo. Las rutas siempre tienen la misma forma:

```
/users/X/<usuario>/albums/<album>/<foto>.png
/users/X/<usuario>/albums/profile.png
```

donde X es una sub-ruta que es igual para todos los usuarios. El programa tiene que funcionar con mínimo 1000 usuarios y la eficiencia es muy importante en la nota.

2 Análisis del problema

Leyendo el enunciado encontramos estas cosas principales: la plataforma, el usuario, la solicitud de amistad, el mensaje, el álbum, la foto, la fecha y un medidor de recursos. La amistad no la hicimos como una clase aparte sino como una relación entre usuarios, y la guardamos en una matriz de adyacencia como sugiere el enunciado.

2.1 Lo que tiene que hacer el programa

N°	Funcionalidad	Que necesita
I	Cargar y guardar datos	Leer todo al iniciar y guardar al cerrar sesión. No sale en el menú.
II	Ingreso y mejores amigos	Buscar el usuario, mostrar 3 fotos de perfil de amigos al azar (con prioridad) y dejar cambiarlas una sola vez.
III	Administrar amistades	Enviar solicitudes (si no son amigos y no hay solicitud pendiente) y aceptar las que llegan.
IV	Visitar perfil de un amigo	Mostrar álbumes públicos o de amigos y recorrer las fotos en forma circular, con like.
V	Sugerencias de amistad	Mostrar K sugerencias (entre 1 y 20) ordenadas por amigos en común y poder enviar solicitud.
VI	Administrar mensajes	Enviar mensajes de máximo 512 caracteres, leer los nuevos y buscar por rango de fechas.
VII	Medir recursos	Mostrar las iteraciones y la memoria usada después de cada funcionalidad.

Tabla 1: Funcionalidades pedidas

2.2 Restricciones

- No se puede usar contenedores de la STL, ni structs, ni librerías externas que resuelvan lo importante. Si se puede usar la librería de aleatorios.
- Las estructuras de datos las tenemos que hacer nosotros y con memoria dinámica.
- No se puede usar herencia.
- Hay que usar constructores de copia, getters, setters, sobrecarga de métodos, mínimo dos operadores sobrecargados, funciones amigas y plantillas.
- El código va separado en archivos .h y .cpp.
- Los archivos no se pueden usar para procesar datos, solo para cargar y guardar.

2.3 Reglas que salen de la lógica

- No se puede mandar solicitud a uno mismo, a un amigo, ni a alguien que ya nos mando solicitud o al que ya le mandamos.
- No se pueden mandar mensajes vacíos ni de más de 512 caracteres, y solo se le puede escribir a amigos.
- Solo se puede visitar el perfil de un amigo y nunca se muestran sus álbumes privados.
- Quitar un like no puede dejar los likes en negativo.
- En la busqueda de mensajes la fecha de inicio no puede ser mayor que la fecha final.

2.4 Lo que no hay que hacer

Registrarse o borrar cuenta, crear o editar álbumes y fotos, eliminar amigos, responder mensajes directamente y validar que cada usuario de un solo like por foto.

3 Como pensamos la solución

La idea es cargar todo a memoria dinámica cuando arranca el programa, trabajar con nuestras propias estructuras y guardar los cambios al cerrar sesión. Cada objeto se crea una sola vez y las demás partes del programa lo usan con punteros o índices, asi no hacemos copias innecesarias.

3.1 Estructuras de datos

Estructura	**Para que**	**Por que la escogimos**
Plantilla `ListaDinamica<T>`	Usuarios, álbumes, fotos, solicitudes y mensajes	Es un arreglo dinámico que duplica su tamaño cuando se llena. Acceder por índice es $O(1)$. Ademas nos sirve para cumplir lo de plantillas.
Arreglo de usuarios ordenado por nombre	Iniciar sesión y buscar usuarios	Con búsqueda binaria es $O(\log N)$, con 1000 usuarios son mas o menos 10 comparaciones.
Matriz de adyacencia de bits (`MatrizAmistad`)	Saber si dos usuarios son amigos y contar amigos en común	Preguntar si son amigos es $O(1)$. Con 1000 usuarios ocupa unos 125 KB en vez de 1 MB si fuera un bool por casilla.
Lista de índices de amigos en cada usuario	Recorrer los amigos	No toca recorrer toda la fila de la matriz si el usuario tiene pocos amigos.
Lista de mensajes enviados y recibidos en cada `Usuario`	Guardar los mensajes	Cada mensaje se crea una sola vez y es del usuario que lo envía. El receptor lo guarda como puntero en sus recibidos, así el estado de leído es el mismo para los dos.

Tabla 2: Estructuras de datos escogidas

3.2 Otras decisiones

- **Rutas:** la sub-ruta fija se guarda una sola vez. Cada `Usuario`, `Album` y `Foto` guarda solo su ruta relativa y la ruta completa se arma al mostrarla. Esto ahorra memoria.
- **Fechas:** la clase `Fecha` sobrecarga `<` y `==`. Por dentro compara un número tipo AAAAMMDDHH, entonces es una sola comparación.
- **Paso por referencia:** los objetos grandes se pasan por referencia constante para no copiarlos.
- **Texto:** usamos `std::string` solo para los textos, como esta en el diagrama.
- **Privacidad del álbum:** la guardamos como un entero (0 público, 1 amigos, 2 privado).

4 Diagrama de clases

El diseño tiene diez clases y ninguna usa herencia. Ocho son las del problema y dos son de apoyo: `ListaDinamica<T>` y `MatrizAmistad`. Las dos de apoyo se dibujan en el diagrama completo.

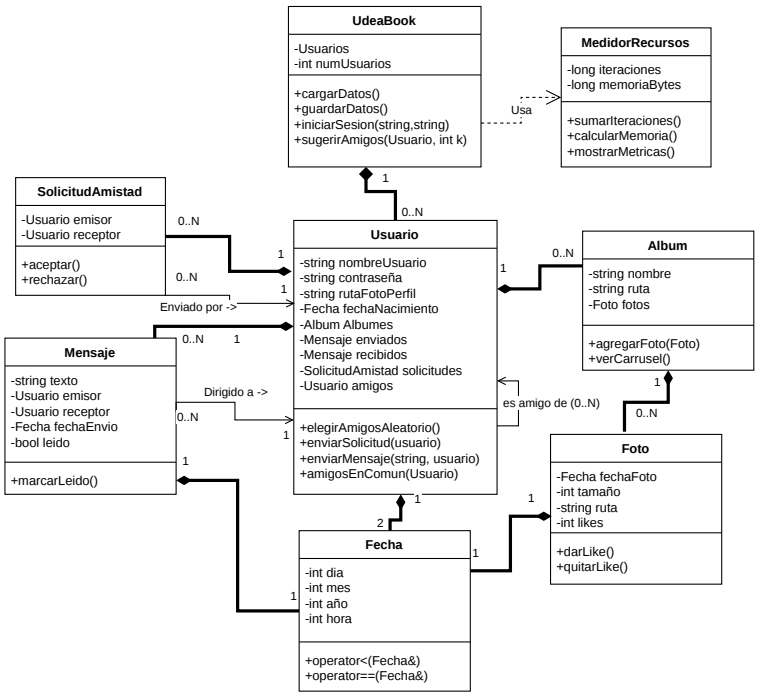


Figura 1: Diagrama de clases

4.1 Que hace cada clase

Clase	Que hace
UdeaBook	Es la clase principal. Carga y guarda datos, hace el inicio de sesión, busca usuarios, calcula sugerencias y muestra el menú.
Usuario	Tiene los datos de la persona, sus amigos, álbumes, mensajes y solicitudes. Hace las operaciones de amistad y mensajes.
SolicitudAmistad	Une al que envía con el que recibe. Cuando se acepta se crea la amistad en los dos sentidos.
Mensaje	Guarda el texto (máximo 512 caracteres), quien lo envió, quien lo recibe, la fecha y si ya se leyo.
Album	Tiene nombre, privacidad y fotos. Controla quien lo puede ver y el carrusel.
Foto	Tiene ruta, fecha, tamaño y likes.
Fecha	Día, mes, año y hora. Se puede comparar con `<` y `==`.
MedidorRecursos	Cuenta las iteraciones y calcula la memoria usada.
`ListaDinamica<T>`	Plantilla de arreglo dinámico que usamos para todas las listas.
MatrizAmistad	Matriz de bits para saber quien es amigo de quien.

Tabla 3: Responsabilidad de cada clase

4.2 Relaciones

Relación	Tipo	Multipl.	Significado
UdeaBook – Usuario	Composición	1 a 0..N	La plataforma crea y borra los usuarios.
UdeaBook – MatrizAmistad	Composición	1 a 1	Una sola matriz para toda la red.
UdeaBook – MedidorRecursos	Dependencia	–	La plataforma lo usa para reportar iteraciones y memoria.
Usuario – Usuario	Asociación reflexiva	0..N a 0..N	"Es amigo de", va en los dos sentidos.
Usuario – SolicitudAmistad	Composición y asociación	1 a 0..N	El receptor es dueño de la solicitud y el emisor se referencia con "Enviada por".
Usuario – Mensaje	Composición y asociación	1 a 0..N	El emisor es dueño del mensaje y el receptor se referencia con "Dirigido a".
Usuario – Album	Composición	1 a 0..N	Cada álbum es de un solo usuario.
Album – Foto	Composición	1 a 0..N	Cada foto es de un solo álbum.
Usuario, Foto, Mensaje – Fecha	Composición	1 a 1 (Usuario: 2)	Fechas de nacimiento e inscripción, captura y envío.
UdeaBook, Usuario, Album – `ListaDinamica<T>`	Dependencia	–	Usan la plantilla para sus listas.

Tabla 4: Relaciones entre clases

5 Algoritmos principales

Aquí explicamos la idea de los algoritmos que no son tan obvios. El código con comentarios va en la segunda entrega.

5.1 Cargar datos e iniciar sesión

1. Leer config.txt y usuarios.txt, creando el arreglo con el tamaño que dice la primera línea.
2. Crear la matriz de bits de N por N en ceros. Por cada línea de amistades.txt se buscan los dos usuarios con búsqueda binaria, se prenden las dos casillas y se agrega cada uno a la lista de amigos del otro.
3. Cargar solicitudes, mensajes, álbumes y fotos y conectarlos con sus usuarios usando punteros.
4. Para iniciar sesión se busca el nombre con búsqueda binaria y se compara la contraseña. Si se equivoca tres veces vuelve al inicio.

5.2 Fotos de perfil al azar con prioridad

A cada amigo le damos un peso según los amigos en común: 2 si tiene 10 o menos, 5 si tiene entre 11 y 20, y 7 si tiene mas de 20. Así se mantiene la proporción $1 : 2.5 : 3.5$ que pide el enunciado pero sin usar decimales.

1. Calcular los amigos en común de cada amigo haciendo un AND entre las filas de la matriz y contando los bits.

2. Sumar todos los pesos, sacar un número al azar entre 0 y esa suma, e ir sumando los pesos hasta pasarse del número. Ese es el elegido.

3. Ponerle peso 0 al elegido para que no salga otra vez y repetir hasta tener 3.

4. Para el cambio (que solo se puede una vez) se quitan los 3 que ya salieron. Si el usuario tiene menos de 6 amigos se completa con los que quedan, sin repetir en el mismo grupo.

5.3 Enviar y aceptar solicitudes

Para enviar, se busca al usuario y se revisa que no sea uno mismo, que no sean amigos (se mira en la matriz) y que no haya una solicitud pendiente entre los dos. Si todo esta bien se crea la solicitud. Para aceptar, se prenden las casillas en la matriz, se agregan los dos a sus listas de amigos y se borra la solicitud.

5.4 Sugerencias de amistad

1. Crear un arreglo de contadores de tamaño N en ceros.

2. Por cada amigo del usuario, recorrer los amigos de ese amigo. Si no es el mismo usuario ni ya es su amigo, se le suma 1 a su contador. Al final el contador es la cantidad de amigos en común.

3. Sacar los K mayores con un arreglo pequeño de tamaño K ordenado (como K es máximo 20 esto es rápido).

4. Mostrarlos y dejar que el usuario le mande solicitud a alguno.

5.5 Carrusel de fotos

Se muestran los álbumes del amigo que sean públicos o de amigos. Al escoger uno, se usa un índice i: siguiente es $(i + 1) \bmod n$ y anterior es $(i - 1 + n) \bmod n$, entonces da la vuelta. También se puede dar o quitar like y salir.

5.6 Mensajes

- **Enviar:** revisar que sea amigo y que el texto tenga entre 1 y 512 caracteres. Se crea el mensaje con la fecha actual y se agrega al final de enviados y recibidos, que siguen quedando ordenados.

- **Leer nuevos:** recorrer los recibidos, mostrar los que no estan leídos con su emisor y marcarlos como leídos.

- **Buscar por rango:** con búsqueda binaria se encuentra el primer mensaje con fecha mayor o igual al inicio y se avanza mientras la fecha sea menor o igual al final.

6 Cosas a tener en cuenta para la implementación

Cada clase va en su propio .h y .cpp, menos la plantilla `ListaDinamica<T>` que va toda en un .h porque así funcionan las plantillas en C++.

- **Liberar memoria:** cada objeto tiene un solo dueño que lo borra. `UdeaBook` borra los usuarios, cada usuario sus álbumes y sus mensajes enviados, y cada álbum sus fotos. Las listas de punteros no borran lo que apuntan.
- **Regla de los tres:** las clases con memoria dinámica tienen constructor de copia, operador de asignación y destructor, para no borrar dos veces lo mismo.
- **Validar entradas:** revisar todo lo que escribe el usuario: opciones del menú, que K este entre 1 y 20, formato de fechas y largo de los mensajes.
- **Datos de prueba:** vamos a generar mínimo 1000 usuarios, con mensajes en más de 6 semanas y amigos con al menos 2 álbumes de 5 fotos, para poder hacer el video.

6.1 Plan de trabajo

1. Hacer `ListaDinamica<T>`, `Fecha` y `MatrizAmistad` y probarlas.
2. Hacer `Usuario`, `Album`, `Foto`, `Mensaje` y `SolicitudAmistad`.
3. Cargar y guardar archivos y hacer el generador de datos.
4. Inicio de sesión y fotos de perfil al azar.
5. Amistades y sugerencias.
6. Carrusel y mensajes.
7. Meter el `MedidorRecursos` en todo.
8. Informe final, video y commits seguidos (mínimo uno cada dos días).